

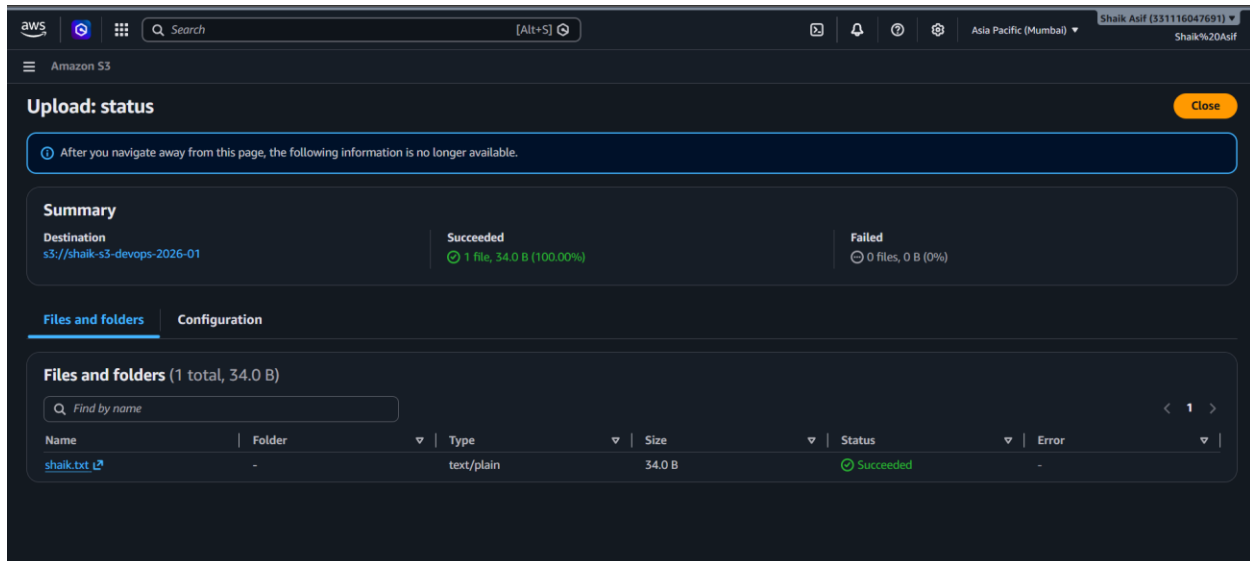
Task 1: Create an S3 Bucket and Upload an Object

Steps Performed

1. Logged in to the AWS Management Console.
2. Opened the Amazon S3 service.
3. Clicked Create bucket.
4. Entered the bucket name:shaik-s3-devops-2026-01
5. Selected the AWS Region:Asia Pacific (Mumbai) - ap-south-1
6. Kept the remaining bucket settings as default.
7. Kept Block all public access enabled.
8. Clicked Create bucket.
9. Opened the newly created bucket.
10. Clicked Upload.
11. Selected the file:shaik.txt
12. Clicked Upload to upload the object.
13. Verified that shaik.txt was successfully displayed inside the S3 bucket.

```
MINGW64:/c/Users/asifs/Downloads
asifs@Asif_Shaik MINGW64 ~/Downloads (master)
$ touch shaik.txt
asifs@Asif_Shaik MINGW64 ~/Downloads (master)
$ vi shaik.txt
```

```
MINGW64:/c/Users/asifs/Downloads
Hi shaik, what are today's plans?
~
~
~
~
~
~
~
```



Result

The S3 bucket shaik-s3-devops-2026-01 was successfully created in the Mumbai (ap-south-1) region, and the shaik.txt object was successfully uploaded to the bucket.

Task 2: Deploy a Static Website in the S3 Bucket

Objective

To deploy a static website using an Amazon S3 bucket by uploading an index.html file, enabling static website hosting, and configuring a bucket policy to allow public access to the website.

Steps Performed

Step 1 – Opened the S3 Bucket

1. Logged in to the AWS Management Console.
2. Opened Amazon S3.
3. Opened the bucket:

shaik-s3-devops-2026-01

Step 2 – Uploaded index.html

1. Opened the Objects tab.

2. Clicked Upload.
3. Selected the index.html file.
4. Clicked Upload.
5. Verified that index.html was present in the bucket.

Step 3 – Enabled Static Website Hosting

1. Opened the Properties tab.
2. Located Static website hosting.
3. Clicked Edit.
4. Selected Enable.
5. Selected Host a static website.
6. Entered index.html as the Index document.
7. Clicked Save changes.

Step 4 – Configured Public Access

To allow users to access the static website, public access settings were configured for the bucket.

1. Opened Permissions.
2. Edited Block public access.
3. Disabled Block all public access.
4. Acknowledged the warning and saved the changes.

Step 5 – Added Bucket Policy

1. In the Permissions tab, opened Bucket policy.
2. Added the following policy:

```
{  
  "Version": "2012-10-17",
```

```

"Statement": [
  {
    "Sid": "PublicReadForWebsite",
    "Effect": "Allow",
    "Principal": "*",
    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::shaik-s3-devops-2026-01/*"
  }
]
}

```

Policy Key Explanation

Policy Key	Purpose
Version	Defines the policy language version
Statement	Contains the permission rules
Sid	Unique name/identifier for the statement
Effect	Specifies whether access is Allow or Deny
Principal	Specifies who can access the resource; * means anyone
Action	Specifies the permitted action; s3:GetObject allows reading objects
Resource	Specifies which S3 objects the policy applies to
arn:aws:s3:::shaik-s3-devops-2026-01/*	Applies the permission to all objects inside the bucket

Step 6 – Tested the Website

1. Returned to Properties → Static website hosting.
2. Copied the Bucket website endpoint.
3. Opened the endpoint in a browser.
4. Verified that the index.html webpage was displayed.

Result

The index.html file was successfully uploaded to the S3 bucket. Static website hosting was enabled, and a bucket policy was configured to allow users to read the website objects.

```
asifs@Asif_Shaik MINGW64 ~/Downloads (master)
$ touch index.html

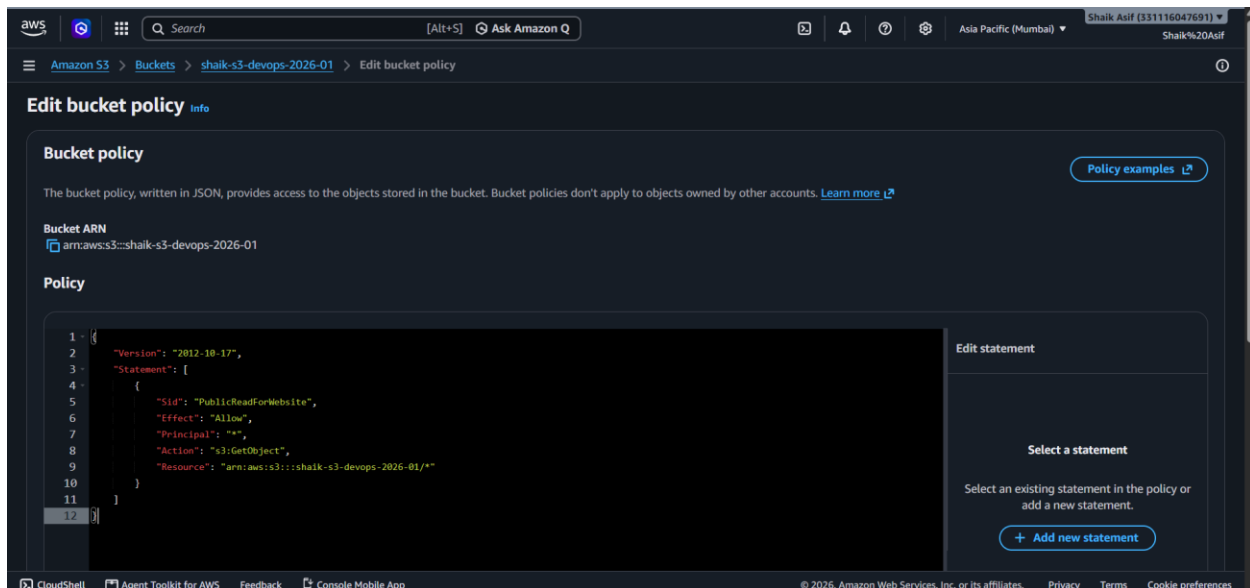
asifs@Asif_Shaik MINGW64 ~/Downloads (master)
$ vi index.html
```

```
MINGW64:/c/Users/asifs/Downloads
<!DOCTYPE html>
<html>
<head>
  <title>Shaik S3 Website</title>
</head>
<body>
  <h1>Welcome to Shaik's S3 Website</h1>
  <p>This website is hosted using Amazon S3.</p>
  <p>I am learning AWS DevOps.</p>
</body>
</html>
```

The screenshot shows the AWS Management Console interface for an Amazon S3 bucket named 'shaik-s3-devops-2026-01'. The 'Objects' tab is active, displaying a list of two objects:

Name	Type	Last modified	Size	Storage class
index.html	html	September 3, 2026, 15:17:39 (UTC+05:30)	228.0 B	Standard
shaik.txt	txt	September 3, 2026, 14:46:15 (UTC+05:30)	34.0 B	Standard

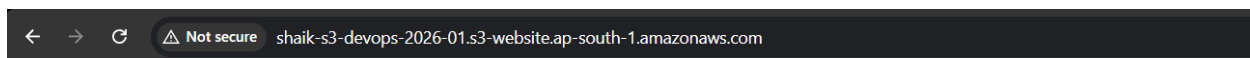
The console also features a sidebar with navigation options (Buckets, Files, Access management and security, Storage management and insights) and a top navigation bar with the AWS logo and search bar.



Welcome to Shaik's S3 Website

This website is hosted using Amazon S3.
I am learning AWS DevOps.

If index.html deleted:



404 Not Found

- Code: NoSuchKey
- Message: The specified key does not exist.
- Key: index.html
- RequestId: GTHJ57ESVEDP0NRV
- HostId: TcXjAu4uKDwOG8ZvsSf6tJbaj35oQXYgFG3bmHgkdJYKA277uUO2MIZuWUbuRkXQj/Hfwjwz9EU=

Conclusion

The static website was successfully deployed using Amazon S3. The index.html file serves as the homepage, while the S3 bucket policy provides the required object read permission for accessing the website through the S3 website endpoint.

Task 3: Enable Cross-Region Replication (CRR) on S3 buckets.

Objective

To configure Amazon S3 Cross-Region Replication (CRR) between two S3 buckets located in different AWS Regions and verify that a newly uploaded object is automatically replicated from the source bucket to the destination bucket.

Architecture

The replication setup was configured as follows:

Source Bucket

- Region: Mumbai
- Region code: ap-south-1
- Versioning: Enabled



S3 Cross-Region Replication (CRR)



Destination Bucket

- Region: Singapore
- Region code: ap-southeast-1
- Versioning: Enabled

Step 1 – Create the Source S3 Bucket

1. Opened the AWS Management Console.
2. Navigated to S3.
3. Created the source bucket.
4. Selected the Mumbai (ap-south-1) Region.

5. Completed the bucket creation.

The source bucket is responsible for storing the original objects.

Step 2 – Create the Destination S3 Bucket

1. Created a second S3 bucket.
2. Selected Singapore (ap-southeast-1) as the Region.
3. Completed the bucket creation.

This bucket acts as the destination for replicated objects.

Step 3 – Enable Versioning on the Source Bucket

1. Opened the source bucket.
2. Navigated to Properties.
3. Located Bucket Versioning.
4. Enabled versioning.

Reason: S3 Cross-Region Replication requires versioning to be enabled on the source and destination buckets.

Step 4 – Enable Versioning on the Destination Bucket

1. Opened the destination bucket.
2. Navigated to Properties.
3. Located Bucket Versioning.
4. Enabled versioning.

Versioning was therefore enabled on both buckets.

Step 5 – Create the Replication Rule

1. Opened the source bucket.
2. Navigated to Management.

3. Selected Replication rules.
4. Clicked Create replication rule.
5. Provided a replication rule name.
6. Selected the option to apply the rule to the required objects.

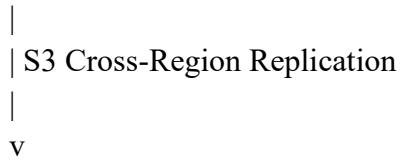
Step 6 – Configure the Destination

For the replication destination:

- Selected the destination bucket.
- Selected the Singapore (ap-southeast-1) bucket.
- Configured the required permissions/IAM role for S3 replication.

The final replication flow was:

Mumbai Source Bucket



Singapore Destination Bucket

Step 7 – Existing Objects Configuration

During replication configuration, S3 provided the option:

Replicate existing objects?

I selected:

No, do not replicate existing objects

This means the replication rule would apply to objects uploaded after the replication configuration was enabled.

The existing shaik.txt file was therefore not used for the replication test.

Step 8 – Upload a New Test File

To test the replication configuration, I created/uploaded a new file:

[asif@techie.txt](#)

The file was uploaded to the source bucket in Mumbai after the CRR rule was enabled.

Step 9 – Verify Replication

After uploading [asif@techie.txt](#) to the source bucket, I checked the destination bucket in Singapore.

The expected result was:

Source – Mumbai

|

| CRR

↓

Destination – Singapore

[asif@techie.txt](#)

The presence of [asif@techie.txt](#) in the destination bucket confirms that the Cross-Region Replication configuration is working.

aws

Search

[Alt+S]

Ask Amazon Q

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik%20Asif

Amazon S3

Buckets

devops-crr-source-asif-2026

devops-crr-source-asif-2026

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems

Access Points

Objects (0)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

Name

Type

Last modified

Size

Storage class

No objects

You don't have any objects in this bucket.

Upload

aws

Search

[Alt+S]

Ask Amazon Q

Asia Pacific (Singapore)

Shaik Asif (331116047691)

Shaik%20Asif

Amazon S3

Buckets

devops-crr-destination-asif-2026

devops-crr-destination-asif-2026

Info

Objects

Metadata

Properties

Permissions

Metrics

Management

File systems - new

Access Points

Objects (0)

Copy S3 URI

Copy URL

Download

Open

Delete

Actions

Create folder

Upload

Objects are the fundamental entities stored in Amazon S3. You can use [Amazon S3 inventory](#) to get a list of all objects in your bucket. For others to access your objects, you'll need to explicitly grant them permissions. [Learn more](#)

Find objects by prefix

< 1 >

Name

Type

Last modified

Size

Storage class

No objects

You don't have any objects in this bucket.

Upload

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik%20Asif

Amazon S3

Buckets

devops-crr-source-asif-2026

Replication rules

Create replication rule

Create replication rule

Info

Use replication rules to define options you want Amazon S3 to apply during replication such as server-side encryption, replica ownership, transitioning replicas to another storage class, and more. Replication also includes any annotations attached to your objects. [Learn more](#)

Replication now includes replicating any annotations attached to your objects, which might increase your costs. To understand the potential cost impact, review your replication rules and annotations usage.

Replication rule configuration

Replication rule name

replicate-to-singapore

Up to 255 characters. In order to be able to use CloudWatch metrics to monitor the progress of your replication rule, the replication rule name must only contain English characters.

Status

Choose whether the rule will be enabled or disabled when created.

☒ Enabled

☐ Disabled

Priority

The priority value resolves conflicts that occur when an object is eligible for replication under multiple rules to the same destination. The rule is added to the configuration at the highest priority and the priority can be changed on the replication rules table.

0

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Shaik%20Asif

Amazon S3

Buckets

devops-crr-source-asif-2026

Replication rules

Create replication rule

Source bucket

Source bucket name

devops-crr-source-asif-2026

Source Region

Asia Pacific (Mumbai) ap-south-1

Choose a rule scope

☐ Limit the scope of this rule using one or more filters

☒ Apply to all objects in the bucket

Destination

Destination

You can replicate objects across buckets in different AWS Regions (Cross-Region Replication) or you can replicate objects across buckets in the same AWS Region (Same-Region Replication). You can also specify a different bucket for each rule in the configuration. [Learn more](#) or see [Amazon S3 pricing](#)

☒ Choose a bucket in this account

☐ Specify a bucket in another account

Bucket name

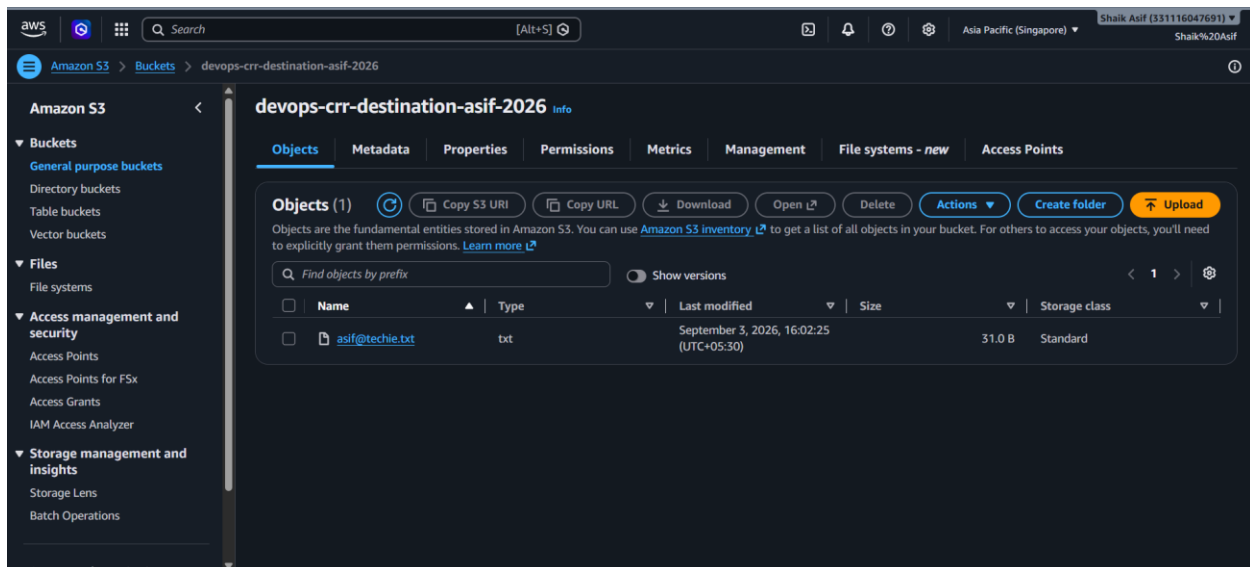
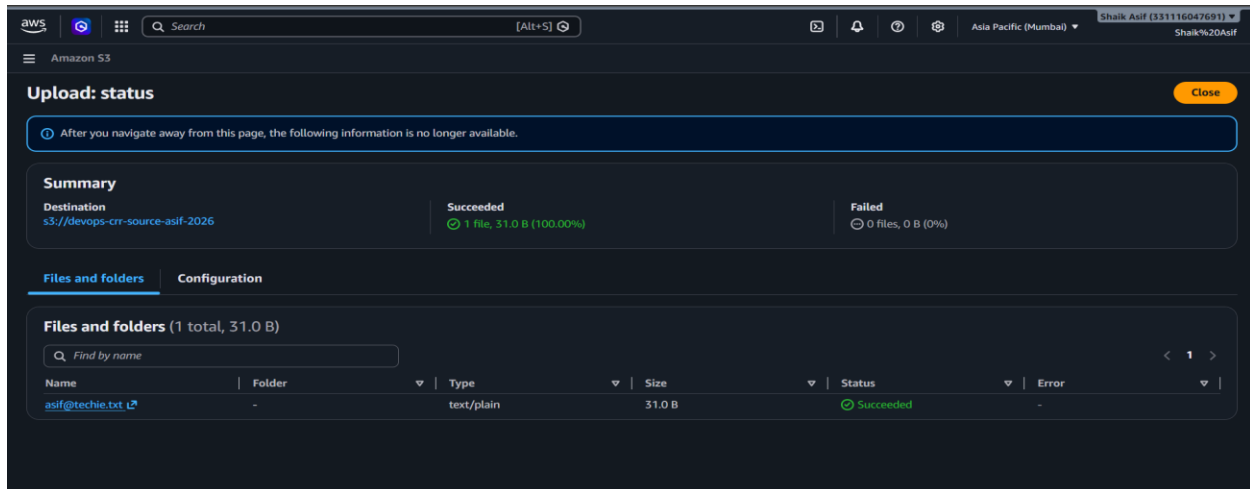
Choose the bucket that will receive replicated objects.

devops-crr-destination-asif-2026

Browse S3

Destination Region

Asia Pacific (Singapore) ap-southeast-4



Conclusion

S3 Cross-Region Replication was successfully configured between the Mumbai source bucket and the Singapore destination bucket. Versioning was enabled on both buckets and a replication rule was created. A new test file, [asif@techie.txt](#), was uploaded to the source bucket to verify the configuration.

The successful appearance of the file in the destination bucket demonstrates that S3 Cross-Region Replication is working correctly, providing data redundancy and supporting disaster recovery and business continuity across AWS Regions.

Task 4 – Configure S3 Bucket Policy to Allow Only Admin User to View Objects

Objective

To configure an Amazon S3 bucket policy so that the designated Admin IAM user can view and read objects stored in the S3 bucket.

Resources Used

- S3 Bucket: devops-admin-policy-asif-2026
- Object: shaik.txt
- Admin IAM User: asifshaik
- Admin User ARN: arn:aws:iam::331116047691:user/asifshaik

Step 1 – Create a New S3 Bucket

1. Open the AWS Management Console.
2. Navigate to S3.
3. Click Create bucket.
4. Enter the bucket name:

devops-admin-policy-asif-2026

5. Select the required AWS Region.
6. Keep Block all public access enabled.
7. Keep the remaining settings as default.
8. Click Create bucket.

The S3 bucket was successfully created for testing the Admin bucket policy.

Step 2 – Upload the Test Object

1. Open the newly created bucket:

devops-admin-policy-asif-2026

2. Go to the Objects tab.

3. Click Upload.
4. Selected the existing file:

shaik.txt

5. Click Upload.

The object was successfully uploaded to the S3 bucket.

The bucket structure was:

devops-admin-policy-asif-2026
└── shaik.txt

Step 3 – Identify the Admin IAM User

Since multiple IAM users were available, the permissions of the users were checked.

The user selected for the Admin role was:

Username: asifshaik

This user had:

- AmazonS3FullAccess
- AmazonEC2FullAccess

The IAM user ARN was obtained from:

IAM → Users → asifshaik → Summary

ARN:

arn:aws:iam::331116047691:user/asifshaik

Step 4 – Enable Console Access for the Admin User

The asifshaik IAM user did not initially have console access.

Therefore:

1. Opened IAM → Users → asifshaik.
2. Opened Security credentials.
3. Configured/managed Console access.
4. Set a new console password for the user.
5. Saved the changes.

The password was not shared or exposed.

This allowed the IAM user to sign in to the AWS Management Console for testing.

Step 5 – Configure the S3 Bucket Policy

1. Opened:

S3 → devops-admin-policy-asif-2026

2. Selected the Permissions tab.
3. Scrolled to Bucket policy.
4. Clicked Edit.
5. Added the following bucket policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowAdminListBucket",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::331116047691:user/asifshaik"
      },
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::devops-admin-policy-asif-2026"
    },
    {
      "Sid": "AllowAdminReadObjects",
      "Effect": "Allow",
      "Principal": {
        "AWS": "arn:aws:iam::331116047691:user/asifshaik"
      },
      "Action": "s3:GetObject",
```

```
    "Resource": "arn:aws:s3:::devops-admin-policy-asif-2026/*"
  }
]
}
```

6. Clicked Save changes.

The bucket policy was successfully added.

Step 6 – Understand the Policy

The policy contains two permissions.

1. s3:ListBucket

s3:ListBucket

This permission allows the Admin user to list/view the objects inside the S3 bucket.

It applies to the bucket itself:

arn:aws:s3:::devops-admin-policy-asif-2026

2. s3:GetObject

s3:GetObject

This permission allows the Admin user to open, read, and download objects from the bucket.

It applies to objects inside the bucket:

arn:aws:s3:::devops-admin-policy-asif-2026/*

3. Principal

The Principal identifies the IAM user who receives the permission:

arn:aws:iam::331116047691:user/asifshaik

Therefore, the policy specifically grants these S3 permissions to the asifshaik user.

Step 7 – Verification

The Admin IAM user asifshaik was configured with console access so that the S3 permissions could be tested.

The expected verification was:

S3



devops-admin-policy-asif-2026

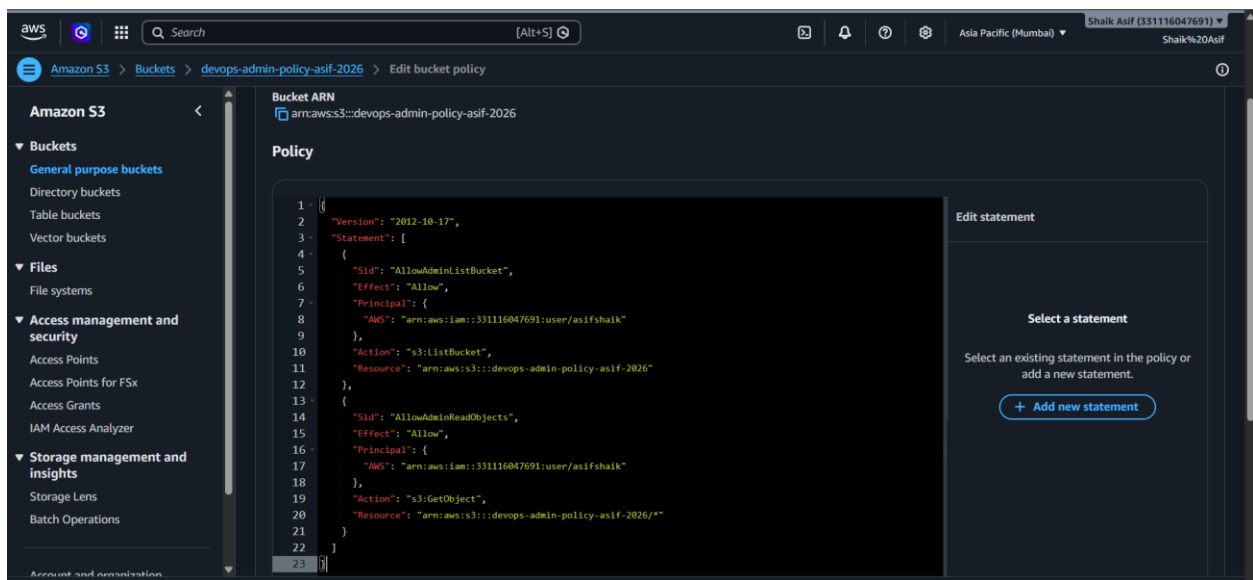


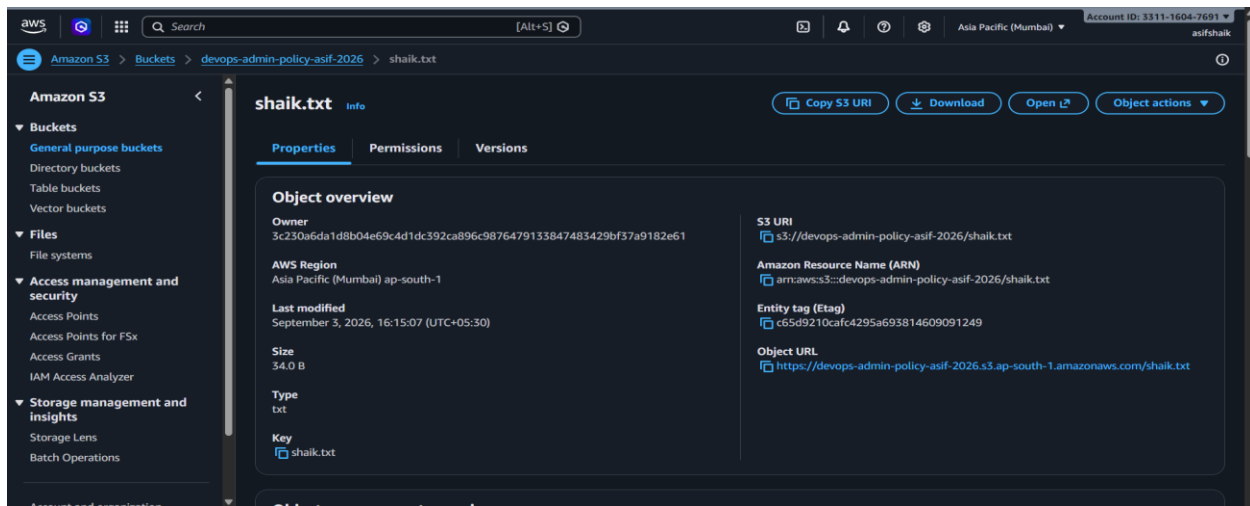
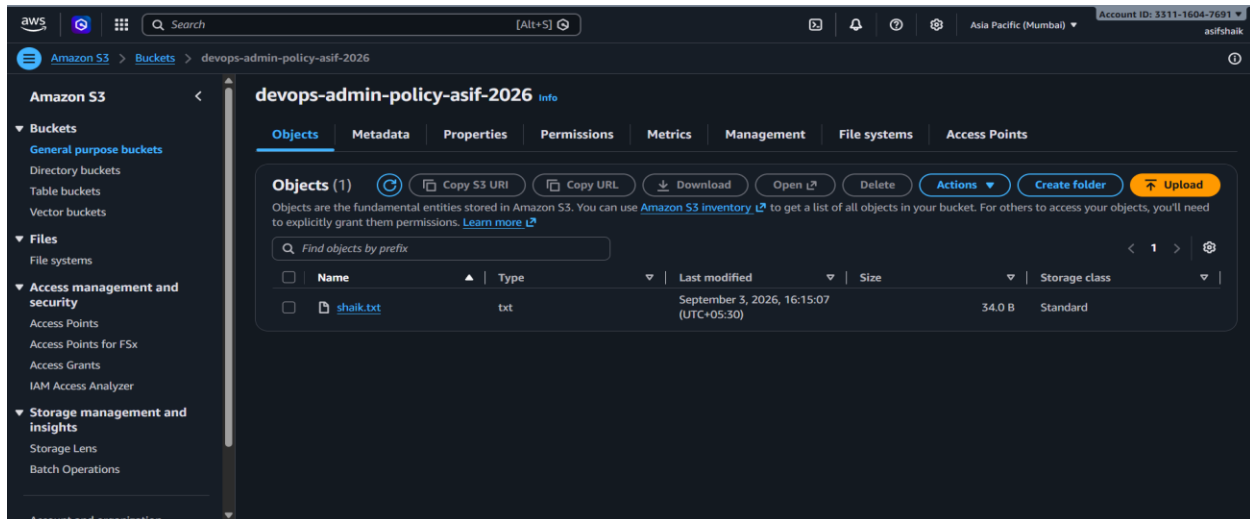
Objects



shaik.txt

The Admin user should be able to see shaik.txt and open/download it.





Conclusion

The S3 bucket devops-admin-policy-asif-2026 was successfully created and the existing shaik.txt object was uploaded. An IAM user named asifshaik was selected as the Admin user and its ARN was used as the Principal in the S3 bucket policy.

The bucket policy grants s3:ListBucket permission to allow the Admin user to view the objects in the bucket and s3:GetObject permission to allow the Admin user to read or download the objects.

Therefore, the S3 bucket policy configuration for Admin access was successfully implemented and is ready for verification.

Task 5: Set up lifecycle policies to automatically transition or delete objects based on specific criteria.

Objective

To configure an Amazon S3 Lifecycle policy that automatically manages objects by transitioning them to a different storage class after a specified period.

Step 1 – Open Amazon S3

1. Logged in to the AWS Management Console.
2. Searched for S3 in the AWS services search bar.
3. Opened Amazon S3.
4. Navigated to the list of available S3 buckets.

Step 2 – Select the S3 Bucket

I selected the bucket created for the previous task:

Bucket Name: devops-admin-policy-asif-2026

This bucket already contained the shaik.txt object.

Step 3 – Open the Management Tab

1. Opened the bucket.
2. Selected the Management tab.
3. Located the Lifecycle rules section.
4. Clicked Create lifecycle rule.

The Lifecycle configuration is used to automatically perform actions on objects based on conditions such as object age.

Step 4 – Provide Lifecycle Rule Name

I entered the lifecycle rule name:

Rule Name: transition-to-standard-ia

The rule name helps identify the purpose of the lifecycle configuration.

Step 5 – Configure the Rule Scope

I configured the lifecycle rule to apply to the objects in the bucket.

The rule was configured without a specific object prefix, so it applies to the objects covered by the rule.

Step 6 – Configure Lifecycle Action

Under the lifecycle actions, I selected the option to:

Transition current versions of objects between storage classes

I then selected:

Storage Class: S3 Standard-IA

Standard-IA (Standard – Infrequent Access) is designed for objects that are accessed less frequently but still need to be available when required.

Step 7 – Configure Transition Period

I configured the transition condition as:

Transition to Standard-IA after 30 days

This means that eligible objects will automatically transition from their current storage class to S3 Standard-IA after they reach the configured age.

The lifecycle action does not happen immediately after creating the rule. AWS applies the lifecycle action automatically when the object meets the configured criteria.

Step 8 – Avoid Object Deletion

I did not configure an expiration/delete action in this rule.

This was done to avoid accidentally deleting the existing shaik.txt object.

The assignment requirement was satisfied by configuring an automatic transition action.

Step 9 – Create the Lifecycle Rule

After reviewing the configuration:

1. Verified the rule name.
2. Verified the scope.
3. Verified the transition action.
4. Verified that the transition period was 30 days.
5. Clicked Create rule.

The lifecycle rule was successfully created.

Step 10 – Verify the Lifecycle Rule

After creating the rule, I returned to:

S3 → devops-admin-policy-asif-2026 → Management → Lifecycle rules

I verified that the rule was displayed.

The rule showed:

- Rule Name: transition-to-standard-ia
- Status: Enabled
- Action: Transition to Standard-IA
- Transition Period: 30 days

This confirmed that the lifecycle configuration was successfully applied to the S3 bucket.

Conclusion

The S3 Lifecycle policy was successfully configured for the bucket devops-admin-policy-asif-2026. A lifecycle rule named transition-to-standard-ia was created and enabled. The rule is configured to automatically transition eligible objects to the S3 Standard-IA storage class after 30 days. This demonstrates automated S3 object lifecycle management and helps optimize storage costs for objects that are accessed less frequently.

Task 6: Push some objects to S3 using the AWS CLI.

Objective

To upload multiple objects to an Amazon S3 bucket using the AWS Command Line Interface (AWS CLI) and verify that the objects were successfully uploaded.

Step 1 – Verify AWS CLI Installation

Opened Git Bash and checked the AWS CLI version using:

```
aws --version
```

The AWS CLI was successfully installed and available for use.

Step 2 – Verify AWS CLI Authentication

Executed:

```
aws sts get-caller-identity
```

The command successfully returned the AWS account and IAM user information.

The CLI was authenticated using the IAM user:

```
asifshaik
```

Step 3 – Verify S3 Bucket Access

Initially, the CLI was configured with another IAM user that did not have sufficient S3 permissions. Therefore, the CLI credentials were changed to the asifshaik IAM user, which had access to the S3 bucket.

Verified access using:

```
aws s3 ls s3://devops-admin-policy-asif-2026/
```

The existing shaik.txt object was displayed, confirming that the IAM user had access to the bucket.

Step 4 – Create Test Objects

Created three test files locally using Git Bash:

```
echo "This is my first S3 CLI object" > file1.txt  
echo "This is my second S3 CLI object" > file2.txt  
echo "This is my third S3 CLI object" > file3.txt
```

Verified the files using:

```
ls -l file1.txt file2.txt file3.txt
```

All three files were successfully created.

Step 5 – Upload Objects Using AWS CLI

Uploaded the first file:

```
aws s3 cp file1.txt s3://devops-admin-policy-asif-2026/
```

Uploaded the second file:

```
aws s3 cp file2.txt s3://devops-admin-policy-asif-2026/
```

Uploaded the third file:

```
aws s3 cp file3.txt s3://devops-admin-policy-asif-2026/
```

The aws s3 cp command successfully copied the local files to the S3 bucket.

Step 6 – Verify Uploaded Objects

Used the following command:

```
aws s3 ls s3://devops-admin-policy-asif-2026/
```

The output displayed the uploaded objects along with the existing shaik.txt object.

Expected objects:

file1.txt
file2.txt
file3.txt
shaik.txt

This confirmed that the objects were successfully uploaded to Amazon S3 using the AWS CLI.

Evidence

The following screenshots can be included as evidence:

1. AWS CLI version output.
2. aws sts get-caller-identity output.
3. S3 bucket access verification.
4. Local files created using Git Bash.
5. Successful aws s3 cp upload output.
6. Final aws s3 ls output showing the uploaded objects.

Result

Three objects were successfully created locally and uploaded to the S3 bucket devops-admin-policy-asif-2026 using AWS CLI.

```

asifs@Asif_Shaik MINGW64 ~
$ aws sts get-caller-identity
{
  "UserId": "AIDAU2GAN5VF7BIS7YVVI",
  "Account": "331116047691",
  "Arn": "arn:aws:iam::331116047691:user/asifshaik"
}

asifs@Asif_Shaik MINGW64 ~
$ aws s3 ls s3://devops-admin-policy-asif-2026/
2026-09-03 16:15:07          34 shaik.txt

asifs@Asif_Shaik MINGW64 ~
$ echo "This is my first S3 CLI object" > file1.txt

asifs@Asif_Shaik MINGW64 ~
$ echo "This is my second S3 CLI object" > file2.txt

asifs@Asif_Shaik MINGW64 ~
$ echo "This is my third S3 CLI object" > file3.txt

asifs@Asif_Shaik MINGW64 ~
$ vi file1.txt

asifs@Asif_Shaik MINGW64 ~
$ ls -l file1.txt file2.txt file3.txt
-rw-r--r-- 1 asifs 197609 31 Sep  3 17:18 file1.txt
-rw-r--r-- 1 asifs 197609 32 Sep  3 17:17 file2.txt
-rw-r--r-- 1 asifs 197609 31 Sep  3 17:18 file3.txt

asifs@Asif_Shaik MINGW64 ~
$ aws s3 cp file1.txt s3://devops-admin-policy-asif-2026/
upload: .\file1.txt to s3://devops-admin-policy-asif-2026/file1.txt

asifs@Asif_Shaik MINGW64 ~
$ aws s3 cp file2.txt s3://devops-admin-policy-asif-2026/
upload: .\file2.txt to s3://devops-admin-policy-asif-2026/file2.txt

asifs@Asif_Shaik MINGW64 ~
$ aws s3 cp file3.txt s3://devops-admin-policy-asif-2026/
upload: .\file3.txt to s3://devops-admin-policy-asif-2026/file3.txt

asifs@Asif_Shaik MINGW64 ~
$ aws s3 ls s3://devops-admin-policy-asif-2026/
2026-09-03 17:19:14          31 file1.txt
2026-09-03 17:19:29          32 file2.txt
2026-09-03 17:19:39          31 file3.txt
2026-09-03 16:15:07          34 shaik.txt

```

The screenshot shows the Amazon S3 console interface. The left sidebar displays the navigation menu with categories like Buckets, Files, Access management and security, and Storage management and insights. The main content area shows the 'devops-admin-policy-asif-2026' bucket. The 'Objects (4)' tab is selected, displaying a table of objects. The table has columns for Name, Type, Last modified, Size, and Storage class. The objects listed are file1.txt, file2.txt, file3.txt, and shaik.txt, all of type 'txt' and stored in the 'Standard' storage class.

Name	Type	Last modified	Size	Storage class
file1.txt	txt	September 3, 2026, 17:19:14 (UTC+05:30)	31.0 B	Standard
file2.txt	txt	September 3, 2026, 17:19:29 (UTC+05:30)	32.0 B	Standard
file3.txt	txt	September 3, 2026, 17:19:39 (UTC+05:30)	31.0 B	Standard
shaik.txt	txt	September 3, 2026, 16:15:07 (UTC+05:30)	34.0 B	Standard

Conclusion

The task was successfully completed by using AWS CLI to upload multiple objects to Amazon S3. The AWS CLI credentials were configured with an IAM user having the required S3 permissions, the objects were created locally, uploaded using the `aws s3 cp` command, and verified using `aws s3 ls`. This demonstrated practical use of AWS CLI for managing S3 objects.

Task 7: Write a Bash script to create an S3 bucket.

Objective

To write and execute a Bash script that creates an Amazon S3 bucket using the AWS CLI.

Step 1 — Create the Bash script

Created the script:

```
nano create_s3_bucket.sh
```

Script used:

```
#!/bin/bash
```

```
BUCKET_NAME="devops-bash-s3-asif-2026"
```

```
REGION="ap-south-1"
```

```
echo "Creating S3 bucket: $BUCKET_NAME"
```

```
aws s3api create-bucket \
```

```
  --bucket "$BUCKET_NAME" \
```

```
  --region "$REGION" \
```

```
  --create-bucket-configuration LocationConstraint="$REGION"
```

```
if [ $? -eq 0 ]; then
```

```
  echo "S3 bucket created successfully: $BUCKET_NAME"
```

```
else
```

```
  echo "Failed to create S3 bucket."
```

```
  exit 1
```

```
fi
```

Step 2 — Give execute permission

```
chmod +x create_s3_bucket.sh
```

Step 3 — Execute the script

```
./create_s3_bucket.sh
```

The script created the bucket:

```
devops-bash-s3-asif-2026
```

in:

ap-south-1 (Mumbai)

Step 4 — Verify the bucket

Used:

```
aws s3api head-bucket --bucket devops-bash-s3-asif-2026
```

The successful output confirmed:

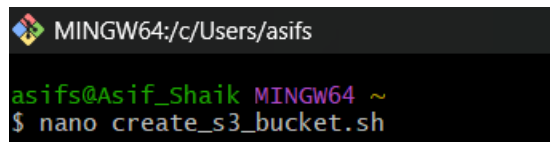
```
"BucketArn": "arn:aws:s3:::devops-bash-s3-asif-2026"
```

```
"BucketRegion": "ap-south-1"
```

Step 5 — Final verification

```
aws s3 ls s3://devops-bash-s3-asif-2026/
```

The bucket was accessible and ready for use.



```
MINGW64:/c/Users/asifs
asifs@Asif_Shaik MINGW64 ~
$ nano create_s3_bucket.sh
```

```
MINGW64:/c/Users/asifs
GNU nano 6.3
#!/bin/bash

BUCKET_NAME="devops-bash-s3-asif-2026"
REGION="ap-south-1"

echo "Creating S3 bucket: $BUCKET_NAME"

aws s3api create-bucket \
  --bucket "$BUCKET_NAME" \
  --region "$REGION" \
  --create-bucket-configuration LocationConstraint="$REGION"

if [ $? -eq 0 ]; then
  echo "S3 bucket created successfully: $BUCKET_NAME"
else
  echo "Failed to create S3 bucket."
  exit 1
fi
```

```
MINGW64:/c/Users/asifs

asifs@Asif_Shaik MINGW64 ~
$ nano create_s3_bucket.sh

asifs@Asif_Shaik MINGW64 ~
$ chmod +x create_s3_bucket.sh

asifs@Asif_Shaik MINGW64 ~
$ ./create_s3_bucket.sh
Creating S3 bucket: devops-bash-s3-asif-2026
{
  "Location": "http://devops-bash-s3-asif-2026.s3.amazonaws.com/",
  "BucketArn": "arn:aws:s3:::devops-bash-s3-asif-2026"
}
S3 bucket created successfully: devops-bash-s3-asif-2026

asifs@Asif_Shaik MINGW64 ~
$ aws s3api head-bucket --bucket devops-bash-s3-asif-2026
{
  "BucketArn": "arn:aws:s3:::devops-bash-s3-asif-2026",
  "BucketRegion": "ap-south-1",
  "AccessPointAlias": false
}

asifs@Asif_Shaik MINGW64 ~
$ aws s3 ls s3://devops-bash-s3-asif-2026/
```

Conclusion

The Bash script was successfully created and executed to automate the creation of an Amazon S3 bucket using the AWS CLI. The bucket devops-bash-s3-asif-2026 was created in the ap-south-1 region and successfully verified using the AWS CLI. This task demonstrates how Bash scripting can be used to automate AWS resource creation and reduce manual effort.

Task 8: Upload a 1 GB file to S3 using the CLI.

Objective

To create a 1 GB test file locally and upload it to an Amazon S3 bucket using the AWS CLI. The purpose is to practice transferring a large object from a local machine to Amazon S3 through the command line.

I created a dummy 1 GB file using the Linux dd command available in Git Bash.

Step 1 — Verify AWS CLI

AWS CLI was already installed and configured on the local Windows machine using Git Bash.

The AWS CLI was being used to interact with Amazon S3.

Step 2 — Use the existing S3 bucket

The bucket created in the previous task was used:

devops-bash-s3-asif-2026

The bucket is located in:

ap-south-1

Step 3 — Create the 1 GB dummy file

The following command was executed:

```
dd if=/dev/zero of=largefile_1GB.bin bs=1M count=1024
```

Explanation

Part	Purpose
------	---------

dd	Command used to create/copy data
if=/dev/zero	Provides zero-filled data
of=largefile_1GB.bin	Name of the output file
bs=1M	Uses 1 MB blocks
count=1024	Creates 1024 blocks

The command produced:

```
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied
```

This confirms that the file was created successfully.

Step 4 — Verify the local file size

The following command was used:

```
ls -lh largefile_1GB.bin
```

Output:

```
-rw-r--r-- 1 asifs 197609 1.0G Sep 3 17:47 largefile_1GB.bin
```

This confirmed that the local file size was approximately 1.0 GB.

Step 5 — Upload the file to S3

The file was uploaded using:

```
aws s3 cp largefile_1GB.bin s3://devops-bash-s3-asif-2026/
```

The CLI returned:

```
upload: ./largefile_1GB.bin to s3://devops-bash-s3-asif-2026/largefile_1GB.bin
```

This confirmed that the upload completed successfully.

Step 6 — Verify the file in S3

The S3 bucket can be verified from the CLI using:

```
aws s3 ls s3://devops-bash-s3-asif-2026/
```

The uploaded object should appear as:

```
largefile_1GB.bin
```

It can also be verified from the AWS Console:

S3 → General purpose buckets → devops-bash-s3-asif-2026 → Objects

The object:

```
largefile_1GB.bin
```

will be displayed in the bucket.

Result

The 1 GB dummy file was successfully created on the local machine and uploaded to the S3 bucket devops-bash-s3-asif-2026 using the AWS CLI.

```
MINGW64:/c/Users/asifs

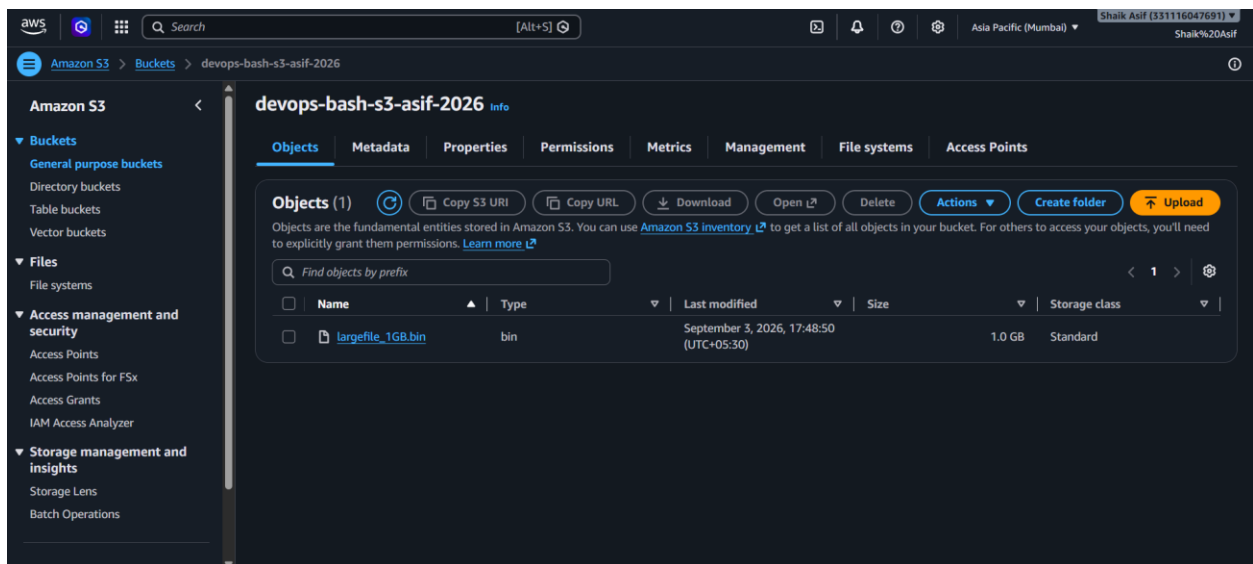
asifs@Asif_Shaik MINGW64 ~
$ dd if=/dev/zero of=largefile_1GB.bin bs=1M count=1024
1024+0 records in
1024+0 records out
1073741824 bytes (1.1 GB, 1.0 GiB) copied, 11.8793 s, 90.4 MB/s

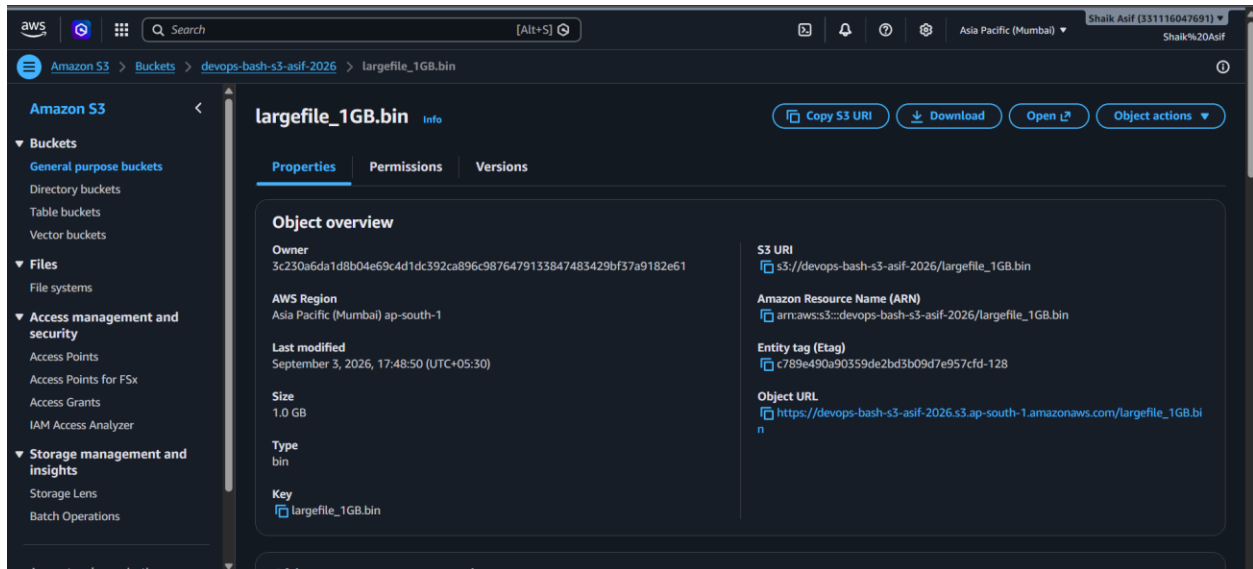
asifs@Asif_Shaik MINGW64 ~
$ ls -lh largefile_1GB.bin
-rw-r--r-- 1 asifs 197609 1.0G Sep  3 17:47 largefile_1GB.bin

asifs@Asif_Shaik MINGW64 ~
$ AC

asifs@Asif_Shaik MINGW64 ~
$ aws s3 cp largefile_1GB.bin s3://devops-bash-s3-asif-2026/
upload: .\largefile_1GB.bin to s3://devops-bash-s3-asif-2026/largefile_1GB.bin

asifs@Asif_Shaik MINGW64 ~
$ aws s3 ls s3://devops-bash-s3-asif-2026/
2026-09-03 17:48:50 1073741824 largefile_1GB.bin
```





Conclusion

The task of uploading a 1 GB file to Amazon S3 using the AWS CLI was successfully completed. A 1 GiB dummy file named `largefile_1GB.bin` was created locally using the `dd` command instead of downloading an actual 1 GB file. The file size was verified locally and then uploaded to the S3 bucket `devops-bash-s3-asif-2026` using the `aws s3 cp` command. The successful upload message confirmed that the object was transferred to S3. This task demonstrated how AWS CLI and Bash/Linux commands can be used to create, manage, and upload large files to Amazon S3 efficiently.